This is another case of using conventions in MVC to reduce the amount of boilerplate you have to write. As always, the conventions are optional. You can also explicitly pass the name of the template to run as a `string` to the `View` method. For example, if the `Index` method in listing 19.6 instead returned `View("ListView")`, the templating engine would look for a template called ListView.cshtml instead. You can even specify the complete path to the view file, relative to your application's root folder, such as `View("Views/global.cshtml")`, which would look for the template at the Views/global .chtml location.

NOTE When specifying the absolute path to a view, you must include both the top-level Views folder and the .cshtml file extension in the path. This is similar to the rules for locating partial view templates.

The process of locating an MVC Razor view is similar to the process of locating a partial view to render, which you learned about in chapter 17. The framework searches in multiple locations to find the requested view. The difference is that for Razor Pages the search process happens only for partial view rendering, as the main Razor view to render is already known; it's the Razor Page's view template.

Figure 19.7 shows the complete process used by the MVC framework to locate the correct View template to execute when a `ViewResult` is returned from an MVC controller. It's possible for more than one template to be eligible, such as if an Index.chstml file exists in both the Home and Shared folders. Similar to the rules for locating partial views, the engine uses the first template it finds.

